Project Report: "Specio" AI Product Search

Version: 1.0.0 **Date:** November 4, 2025 **Status:** Live Development Build

1. Project Overview

1.1. Project Objective

The "Specio" project is a full-stack, AI-native web application designed to provide a highly accurate, semantic search experience for a product catalog (specifically home and office furniture).

Unlike traditional keyword search, Specio uses AI-generated vector embeddings to understand the *meaning* and *context* of a user's query. This allows users to search using natural language (e.g., "a comfy chair for my office") and receive relevant results.

The system is augmented with Generative AI to provide unique, witty descriptions for products and is secured using modern, token-based authentication.

1.2. Core Technologies

Domain	Technology	Purpose
Frontend	React (Vite), TypeScript, Tailwind CSS	A high-performance, modern, and aesthetically-pleasing user interface.
Backend	Python, FastAPI, Pydantic	A high-speed, asynchronous API server for handling requests.
AI (Search)	`sentence-transformers` , Pinecone	Vector embedding generation and a dedicated vector database for high-speed similarity search.
AI (Generative)	`langchain-groq`	Generating dynamic, witty product descriptions in real-time.
Data Vis	`chart.js` , `react-chartjs-2`	Displaying analytics on the frontend dashboard.
Authentication	Clerk (React SDK, Python SDK)	Full-stack user authentication, session management, and JWT (JSON Web Token) verification.

2. Final System Architecture

The application operates on a decoupled, full-stack model.

1. **Frontend (React Client):** The user interacts with the React application running in their browser.
2. **Authentication (Clerk):**

- The frontend uses `<ClerkProvider>` and protected routes (`<SignedIn>`) to redirect unauthenticated users to a Clerk-hosted sign-in page.
- Upon successful login, Clerk provides a short-lived JWT to the frontend.
- The frontend uses the `useAuth()` hook to attach this JWT as a `Bearer` token to the `Authorization` header of every API request.

3. **Backend (FastAPI Server):**
   - The backend receives the request. Every protected endpoint (all of them) uses a FastAPI "Dependency" (`Depends(get_auth_user)`).
   - This dependency, defined in `core/auth.py` , uses the `clerk-python-sdk` and the backend's `CLERK_SECRET_KEY` to verify the JWT.
   - If the token is valid, the API endpoint code is executed. If not, a `401 Unauthorized` error is returned.
4. **AI Service Layer (`ai_service.py`):**
   - **Search:** For search requests, the user's query is passed to the embedding model (`all-mpnet-base-v2`) to create a new 768-dimension vector.
   - **Filter:** Filter parameters (brand, price) are built into a Pinecone metadata filter dictionary.
   - **Query:** The vector and filter dictionary are sent to the **Pinecone** index, which returns a list of the most similar product IDs.
   - **Generate:** Product titles are sent to **Groq's LLaMA 3.1** model to generate new descriptions.
5. **Data Layer (`analytics.py` , `filters.py`):**
   - For analytics or filter data, the API reads directly from the static `products.csv` file using `pandas` to generate on-the-fly statistics or brand lists.

3. Final File Structure

This is the complete file structure of the final application.

```
product-recommendation-app/
├── .gitignore
├── README.md
│
├── backend/
│   ├── .env                    # Stores all backend secret keys (Pinecone, Groq, Clerk
│   ├── requirements.txt
│   └── app/
│       ├── __init__.py
│       ├── main.py             # FastAPI app entry point, includes routers
│       ├── api/
│       │   ├── __init__.py
│       │   ├── analytics.py    # Defines the /api/v1/analytics/summary endpoint
│       │   ├── filters.py      # Defines the /api/v1/filters/brands endpoint
│       │   └── recommend.py    # Defines the /api/v1/recommend/search endpoint
│       ├── core/
```

```
│       │   ├── __init__.py
│       │   ├── auth.py         # Defines the get_auth_user dependency for Clerk
│       │   └── config.py       # Pydantic Settings model to load .env
│       └── services/
│           ├── __init__.py
│           └── ai_service.py   # Singleton service to manage AI models (Pinecone, Groq
│       └── schemas/
│           ├── __init__.py
│           └── product.py      # Pydantic schemas (ProductSearchResult, ProductAnalyti
│
├── data/
│   └── products.csv            # The raw product dataset
│
├── frontend/
│   └── product-recommendation-app/
│       ├── .env                # Stores frontend publishable keys (VITE_CLERK_PUBLISHA
│       ├── index.html          # Main HTML entry point
│       ├── package.json        # Frontend dependencies and scripts
│       ├── tailwind.config.js  # Tailwind CSS configuration
│       ├── postcss.config.js   # PostCSS configuration (for Tailwind v3)
│       ├── vite.config.ts      # Vite configuration
│       ├── tsconfig.json       # TypeScript project config
│       ├── tsconfig.app.json   # TypeScript app-specific config
│       ├── tsconfig.node.json  # TypeScript node-specific config
│       └── src/
│           ├── index.css       # Main CSS entry (contains @tailwind directives)
│           ├── main.tsx        # React app entry (Renders <ClerkProvider>, <BrowserRou
│           ├── App.tsx         # Main router setup (<Routes>, <Route>, <SignedIn>)
│           ├── vite-env.d.ts   # TypeScript definitions for Vite (e.g., SVG imports)
│           ├── assets/
│           │   └── Logo_Spacio.svg # Our logo file
│           ├── components/
│           │   ├── Layout.tsx      # Main layout (Navbar + Outlet)
│           │   ├── Navbar.tsx      # Top navigation bar (with overlapping logo, links,
│           │   ├── ProductCard.tsx   # Displays a single product
│           │   └── SkeletonCard.tsx  # Loading placeholder for the product card
│           └── pages/
│               ├── AnalyticsPage.tsx # Dashboard with charts (Bar, Pie)
│               ├── ChatPage.tsx      # Main search/filter/results UI
│               └── LoginPage.tsx     # (Deprecated, Clerk handles this now)
│
└── notebooks/
    ├── Data_Analytics_Notebook.ipynb   # Jupyter notebook for ETL, embedding, and Pine
    └── Model_Training_Notebook.ipynb   # Jupyter notebook for experimenting with CV (C
```

4. Project Walkthrough & Journal

This section details the chronological development process, including problems encountered and solutions implemented.

Phase 1: Initial Setup & Styling Nightmare

We began with a full set of unstyled, unconnected files. The first major hurdle was simply getting the frontend to look correct.

- **Problem:** The frontend rendered as plain, unstyled HTML (`image_b4119d.png`).
- **Solution:** Identified that the main CSS file (`index.css`) was missing the core Tailwind directives (`@tailwind base` , etc.).
- **Problem:** After adding directives, styling was still broken.
- **Solution:** Discovered that the project was missing `tailwind.config.js` and `postcss.config.js` . These files are mandatory for Tailwind to scan the code and generate the required CSS.
- **Problem:** Attempting to generate these files with `npx tailwindcss init -p` failed with an `npm error could not determine executable to run` .
- **Problem:** We then encountered a cascade of versioning errors related to Tailwind v4, Vite, and PostCSS. The ecosystem had very recent, breaking changes.
- **Decision (Key Technical Pivot):** We decided to **abandon Tailwind v4** as it was unstable. We reverted to a known-stable stack:
  - `npm uninstall tailwindcss@^4.0.0`
  - `npm install -D tailwindcss@^3.0.0 postcss@^8.0.0 autoprefixer@^10.0.0`
- **Solution:** We manually created `tailwind.config.js` (populating the `content` array to scan `*.tsx` files) and `postcss.config.js` (registering `tailwindcss` and `autoprefixer` as plugins). After a server restart and cache clear, styling finally appeared.

Phase 2: UI/UX Refinement & Branding

With styling working, we focused on making the app professional.

- **Goal:** Elevate the "boring" UI (`image_90c738.png`) to a "professional grade" application.
- **Branding:** We workshopped a project name, moving from "AI Recommender" to "FurnishFind," and finally settling on the more creative and psychological **"Specio"** (from Latin *specere*, "to see," and "spec" for specifications).
- **Logo Design:** We designed a logo for Specio and iterated until we had a transparent SVG.
- **Navbar Overhaul (`Navbar.tsx`):**
  - **Problem:** The user wanted a large logo (`h-24`) in a small navbar (`h-16`).
  - **Solution:** We implemented an **overlapping logo design**. We made the navbar container `relative` and the `<img>` tag `absolute` , positioning it with `top-1/2 -translate-y-1/2` . We added an invisible placeholder `div` to maintain the layout and prevent links from sliding under the logo.
- **Layout Redesign (`ChatPage.tsx`):**
  - Converted the results list into a responsive `grid` (`grid-cols-1 md:grid-cols-2 lg:grid-cols-3`).
  - Upgraded `ProductCard.tsx` from a horizontal to a vertical layout.

- Crucially, updated `SkeletonCard.tsx` to mirror the new vertical layout, **preventing "layout jank"** (the page jumping when content loads).
- Added abstract background gradient "blobs" for visual appeal, using `z-index` to layer them behind the content.
- **Problem:** The blobs on `ChatPage` were covering the `Navbar` links.
- **Solution:** Added `relative z-20` to the `Navbar.tsx` `<nav>` element, lifting it above the `z-10` content plane of the `ChatPage`.

Phase 3: Core AI Functionality Upgrade

- **Problem:** The user noted the search results were not high-quality (`image_906ce2.jpg`).
- **Diagnosis:** The AI's "understanding" of products was weak. It was based only on `title` and `description` (which was often missing), and used a small, fast model (`all-MiniLM-L6-v2` , 384 dimensions).
- **Solution:** We performed a major upgrade in `Data_Analytics_Notebook.ipynb` :
  1. **Richer Embeddings:** We changed the text embedding to include the `brand` and `main_category` (e.g., `"Title. Brand: X. Category: Y. Description..."`).
  2. **Better Model:** We upgraded to the `sentence-transformers/all-mpnet-base-v2` model, which is larger and more nuanced (768 dimensions).
  3. **Data Re-Upload:** This required re-running the entire notebook to generate new 768-dimension vectors and **uploading them to a new Pinecone index** configured for 768 dimensions.
  4. The `ai_service.py` and backend `.env` file were updated to point to this new model and index.

Phase 4: Feature Expansion - Filtering

- **Goal:** Allow users to filter results by brand and price.
- **Backend Solution:**
  1. Created a new file `api/filters.py` with a `/api/v1/filters/brands` endpoint that reads `products.csv` with Pandas and returns a unique, sorted list of brands.
  2. Updated `main.py` to include this new `filters.router` .
  3. Updated `api/recommend.py` : Modified the `QueryIn` schema to accept `brand_filter` , `min_price` , and `max_price` .
  4. In the `search` function, we implemented logic to build a Pinecone `filter` dictionary (e.g., `{"brand": {"$eq": "BrandName"}, "price": {"$gte": 50, "$lte": 200}}`) and pass it to the `pinecone.query()` method.
- **Frontend Solution (`ChatPage.tsx`):**
  1. Added `useState` hooks for `brandFilter` , `minPrice` , and `maxPrice` .

2. Added a `useEffect` hook to call our new `/filters/brands` endpoint on page load and populate a `brands` state array.
3. Replaced the brand text input with a `<select>` dropdown, which dynamically renders `<option>` tags from the `brands` state.
4. Modified the `handleSearch` function to include the filter states in the `axios.post` payload.

Phase 5: Security & Dependency Hell

- **Goal:** Secure the entire application.
- **Frontend Solution:**
  1. `npm install @clerk/clerk-react`.
  2. Added `VITE_CLERK_PUBLISHABLE_KEY` to `.env`.
  3. Wrapped `main.tsx` in `<ClerkProvider>`.
  4. Rewrote `App.tsx` routing: The root path (`/`) now renders `<SignedIn><Layout /></SignedIn>` or `<SignedOut><Navigate to="/sign-in" /></SignedOut>`, effectively protecting all main pages. Public routes for `/sign-in` and `/sign-up` were added.
  5. Added `<UserButton>` to `Navbar.tsx` for profile/logout.
  6. **Crucially:** Updated `ChatPage.tsx` and `AnalyticsPage.tsx`. We imported the `useAuth` hook and wrapped all `axios` calls to include the auth token:

```
const { getToken } = useAuth();
const token = await getToken();
await axios.get(url, { headers: { Authorization: `Bearer ${token}` } });
```